

OK! Comentários no
seu e-mail apropriado.

PROJETO 3: PROJETO DE UM S.O. (BOS - BASIC OPERATING SYSTEM)

Gustavo Borges Peres da Silva¹, Michael P. C. Silva¹, Rafael S. Melo¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

{gustavo.silva, michael.silva}@discente.ufcat.edu.br,
rafinhalelograma@hotmail.com

Objetivos

- I/O: teclado;
- Tratamento de interrupção;
- Criação de um shell muito simples.

1. Tarefa 1: implementação de E/S para o teclado

1.1. Máquina utilizada

Para este projeto, será utilizado a mesma máquina hospedeira Windows 11 apresentada no tópico “1.1 Descrição” do projeto 1, cujo a mesma possui também um guest/convidado com a distro Debian Linux devido sua disponibilidade e licenciamento deste sistema que será utilizada para o andamento da tarefa 1, responsável por preparar o sistema e criar o primeiro kernel. Vale lembrar também, que nessa máquina possui dois usuários (so2 e vboxuser), portanto, o usuário que iremos utilizar é o vboxuser pois possui privilégios administrativos necessários para darmos continuidade.

1.2. Implementação de um Shell simples

Antes de iniciarmos, iremos abrir o nosso arquivo build.sh e adicionaremos um novo parâmetro à linha do gcc: -ffreestanding (ao final da linha). Essa flag vai evitar que o compilador use qualquer função oferecida pelo sistema hospedeiro, a não ser aquelas que definirmos durante a prática.

```
#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o -ffreestanding
gcc -m32 -c -w screen.c -o screen.o -ffreestanding
gcc -m32 -c -w string.c -o string.o -ffreestanding
gcc -m32 -c -w system.c -o system.o -ffreestanding
ld -m elf_i386 -T link.ld -o basic/boot/kernel.bin kasm.o kc.o screen.o string.o system.o
```

Código 1 – Conteúdo do arquivo script build.sh.

Agora iremos implementar uma sugestão realizada pelo professor, que se trata-se de

uma melhoria da função **print()** em **screen.h** que é a criação de uma variável **length** do tipo **uint8** para armazenar o valor de **strlen(ch)+1**. Com isso, evita-se que a cada execução do laço for se chame novamente a função **strlen()** para que essa obtenha o mesmo valor.

Por conta dessa sugestão, tivemos de adicionar as variáveis em **screen.h** e no código **screen.c** atualizamos a função **print()** conforme abaixo.

```
#ifndef SCREEN_H
#define SCREEN_H

#include "types.h"
#include "system.h"
#include "string.h"

extern int32 cursorX;
extern int32 cursorY;

void printch(char c);
void print(string ch);

#endif
```

Código 2 – Atualização do cabeçalho **screen.h**.

```
#include "basic/include/screen.h"

int32 cursorX = 0, cursorY = 0; // Posição do cursor na tela
const uint8 sw = 80, sh = 25, sd = 2; // Dimensões da tela (largura, altura, profundidade)

// Limpa a tela de uma linha específica até outra
void clearLine(uint8 from, uint8 to) {
    uint16 i = sw * from * sd;
    string vidmem = (string)0xb8000;
    for (i; i < (sw * (to + 1) * sd); i++) {
        vidmem[i] = 0x0; // Valor (caractere) 0
        vidmem[i+1] = 0x0F; // Cor branca sobre fundo preto
    }
}

// Atualiza a posição do cursor na tela
void updateCursor() {
    unsigned temp;
    temp = cursorY * sw + cursorX;
    outportb(0x3D4, 14);
    outportb(0x3D5, temp >> 8);
    outportb(0x3D4, 15);
    outportb(0x3D5, temp);
}

// Limpa toda a tela
void clearScreen() {
    clearLine(0, sh-1); // Chama clearLine p/ limpar da linha 0 até n (sh-1)
    cursorX = 0; // Reseta a posição do cursor na largura
    cursorY = 0; // Reseta a posição do cursor na altura
    updateCursor(); // Atualiza a posição do cursor
}
```

```

// Desloca conteúdo da tela para cima, remove 'lineNumber' linhas na parte superior
void scrollUp(uint8 lineNumber) {
    string vidmem = (string)0xb8000;
    uint16 i = 0;
    for (i; i < sw * (sh - 1) * sd; i++) {
        vidmem[i] = vidmem[i + sw * sd * lineNumber]; // Move o conteúdo da tela p/ cima
    }

    clearLine(sh - 1 - lineNumber, sh - 1); // Limpa linhas recém-criadas na parte inferior
    if ((cursorY - lineNumber) < 0) {
        cursorY = 0;
        cursorX = 0;
    } else {
        cursorY -= lineNumber; // Atualiza a posição do cursor na altura
    }
    updateCursor(); // Atualiza a posição do cursor
}

// Verifica se o cursor atingiu a última linha da tela, e se sim, rola a tela para cima
void newLineCheck() {
    if (cursorY >= sh-1) {
        scrollUp(1); // Chama scrollUp p/ rolar a tela p/ cima removendo uma linha
    }
}

// Imprime um caractere na posição atual do cursor
void printch(char c) {
    string vidmem = (string)0xb8000;
    switch(c) {
        case (0x08): // Retrocesso (BACKSPACE)
            if(cursorX > 0) {
                cursorX--; // Move cursor p/ posição anterior na largura
                vidmem[(cursorY * sw + cursorX)*sd] = 0x00; // Define o caractere como nulo
            } break;
        case (0x09): // Representa a tecla TAB
            cursorX = (cursorX + 8) & ~(8 - 1); // Move cursor p/ a próxima posição TAB
            break;
        case ('\r'): // Retorno de Carro
            cursorX = 0; // Move o cursor para a primeira posição na largura
            break;
        case ('\n'): // Nova Linha
            cursorX = 0; // Move o cursor para a primeira posição na largura
            cursorY++; // Move o cursor para a próxima linha na altura
            break;
        default:
            vidmem[((cursorY * sw + cursorX)) * sd] = c; // Define o caractere na tela
            vidmem[((cursorY * sw + cursorX)) * sd + 1] = 0x0F; // Define a cor do caractere
            cursorX++; // Move o cursor para a próxima posição na largura
            break;
    }
    if (cursorX >= sw) {
        cursorX = 0; // Se ultrapassar a largura, move o cursor para a próxima linha
        cursorY++; // na altura
    }

    newLineCheck(); // Verifica se o cursor atingiu a última linha da tela
    updateCursor(); // Atualiza a posição do cursor
}

// Imprime uma string chamando repetidamente printch para cada caractere na string
void print(string ch) {

```

```

uint8 length = strlen(ch) + 1; // Calcula o comprimento uma vez
uint16 i = 0;

for (i; i < length; i++) {
    printch(ch[i]); // Chama printch para cada caractere na string
}

```

Código 3 – Conteúdo do arquivo screen.c.

Com a sugestão implementada, agora daremos continuidade com a tarefa, cujo, para a implementação de um shell simples, teremos que criar a primeira função de entrada de dados, similar a função scanf() ou gets() em C. A função a ser criada se chama readStr() para a leitura de uma string. Além desta função, também será criada uma função acessório chamada strEql() a qual irá comparar duas strings. A função strEql() criada no cabeçalho (Código 4), será implementada em string.c (Código 5) cujo descrevemos o funcionamento da mesma através dos comentários.

```

#ifndef STRING_H
#define STRING_H
#include "types.h"

uint16 strlenh(string ch); // Função para calcular o comprimento de uma string
uint8 strEql(string ch1, string ch2); // Função para comparar duas strings

#endif

```

Código 4 – Conteúdo do cabeçalho string.h.

```

#ifndef STRING_H
#define STRING_H
#include "types.h"

uint16 strlenh(string ch); // Função para calcular o comprimento de uma string
uint8 strEql(string ch1, string ch2); // Função para comparar duas strings

#endif

```

Código 5 – Conteúdo do arquivo string.c.

Para a criação da função readStr(), esta deve ser inserida dentro do arquivo kb.h, no diretório “include” (Código 6). E sua função foi implementada separadamente no código 7 para uma melhor organização.

```

vboxuser@Debian12:~/Projeto_2/so$ ls
basic      kasm.o      kernel.c    make.sh     screen.o    system.c
basic.iso  kc.o        kernel.o    run.sh      string.c    system.o
build.sh   kernel.asm  link.ld     screen.c    string.o
vboxuser@Debian12:~/Projeto_2/so$ nano basic/include/kb.h
vboxuser@Debian12:~/Projeto_2/so$ nano kb.c

```

Figura 1: Criando arquivo de cabeçalho kb.h e arquivo kb.c.

```

#ifndef KB_H
#define KB_H
#include "types.h"
#include "system.h"
#include "string.h"

// Função para traduzir os códigos de rastreamento de teclado para ASCII
string readStr();

#endif

```

Código 6 – Conteúdo do cabeçalho kb.h.

```

#include "basic/include/kb.h"

string readStr() {
    char buff[256];
    string buffstr = buff;
    uint8 i = 0;
    uint8 reading = 1;
    while(reading){
        if(inportb(0x64) & 0x1){
            switch(inportb(0x60)){
                /*case 1:
                    printch((char)27);
                    buffstr[i] =(char)27;
                    i++;
                    break;*/

                case 2:
                    ...

            }
        }
    }
    buffstr[i] = 0;
    return buffstr;
}

```

Código 7 – Conteúdo parcial do arquivo kb.c.

O objetivo desta função é traduzir os códigos de rastreamento de teclado (scancode) para código American Standard Code for Information Interchange (ASCII). Essa conversão é realizada no switch() que aparece no Código 7. Uma pequena amostra de como essa conversão deve ser realizada é apresentada dentro do switch(), para o scancode 1 (representando a tecla ESC) cujo a mesma está apresentada em comentário. Além disso, utilizaremos a tabela completa de scancodes disponibilizada pelo site Milliseconds [2] para termos a base de até quando teremos de implementar em nosso código que no caso foi proposto criar as entradas de conversão até a entrada 57 da tabela de scancodes.

Vale ressaltar que o **scancode** fornecido é para teclados americanos. E o teclado utilizado para este projeto é teclado mecânico HyperX HX 60 Alloy Origins AQU, Layout 60%, **US – americano** conforme a Figura 2. Os demais membros do grupo tiveram de testar outros layouts de teclados.



Figura 2: Teclado utilizado para o projeto.

Além disso, percebemos que no código de exemplo fornecido pelo professor (Codigo 7) a função de leitura é um “laço infinito”. Sendo assim, pensamos em utilizar uma condição de parada bem simples, cujo, ao pressionar ESC, atribuímos um valor igual a zero para a variável reading fazendo com que a leitura não seja prosseguida, logo, resultando-se na parada do laço.

Por outro lado, vale observar também que, para ler dados do teclado, é necessário ler dados de duas portas diferentes: 0x64 e 0x60.

- O endereço 0x64 permite ler o status do teclado, para saber quando uma tecla é pressionada.
- O endereço 0x60 permite a leitura do “valor” da tecla pressionada.

O `inportb(0x64) & 0x1` indica se uma tecla foi pressionada. Esta operação bitwise resultará em 1 para qualquer scancode válido (não existe scancode com valor 0). `inportb(0x60)` fornece o valor do scancode propriamente dito, o qual será convertido para código ASCII.

Conquanto, após toda essa análise, partimos para a finalização do nosso código 7 inserindo o restante das opções no `switch()` e condição de parada conforme podemos observar em nosso código completo/atualizado abaixo (Código 8).

```
#include "basic/include/kb.h"

string readStr() {
    char buff[256];
    string buffstr = buff;
    uint8 i = 0;
    uint8 reading = 1;
    while(reading){
        //0x64 = endereço da porta de status do controlador
        if(inportb(0x64) & 0x1){
            // 0x60 = endereço da porta de dados do controlador
            switch(inportb(0x60)){
                case 1: //ESC
```

```

        printch((char)27);
        buffstr[i] = (char)27;
        reading = 0; // Encerra a leitura/loop
        break;
    case 2: //1
        printch('1');
        buffstr[i] = '1';
        i++;
        break;
    case 3: //2
        printch('2');
        buffstr[i] = '2';
        i++;
        break;
    case 4: //3
        printch('3');
        buffstr[i] = '3';
        i++;
        break;
    case 5: //4
        printch('4');
        buffstr[i] = '4';
        i++;
        break;
    case 6: //5
        printch('5');
        buffstr[i] = '5';
        i++;
        break;
    case 7: //6
        printch('6');
        buffstr[i] = '6';
        i++;
        break;
    case 8: //7
        printch('7');
        buffstr[i] = '7';
        i++;
        break;
    case 9: //8
        printch('8');
        buffstr[i] = '8';
        i++;
        break;
    case 10: //9
        printch('9');
        buffstr[i] = '9';
        i++;
        break;
    case 11: //0
        printch('0');
        buffstr[i] = '0';
        i++;
        break;
    case 12: //-
        printch('-');
        buffstr[i] = '-';
        i++;
        break;
    case 13: // =

```

atual na string

```
        printch('=');
        buffstr[i] = '=';
        i++;
        break;
case 14: // BACKSPACE
        // Imprime char de controle '\b' para apagar na tela
        printch('\b');
        // Verifica se há chars na string antes de retroceder
        if (i > 0) {
            i--; // Retrocede indice na string
            buffstr[i] = 0; // Remove último char ao atribuir zero à posição
        }
        break;
case 15: //TAB
        printch('\t'); // Imprime o caractere de tabulação
        buffstr[i] = '\t'; // Armazena o caractere na string
        i++;
        break;
case 16: //q
        printch('q');
        buffstr[i] = 'q';
        i++;
        break;
case 17: //w
        printch('w');
        buffstr[i] = 'w';
        i++;
        break;
case 18: //e
        printch('e');
        buffstr[i] = 'e';
        i++;
        break;
case 19: //r
        printch('r');
        buffstr[i] = 'r';
        i++;
        break;

case 20: //t
        printch('t');
        buffstr[i] = 't';
        i++;
        break;
case 21: //y
        printch('y');
        buffstr[i] = 'y';
        i++;
        break;
case 22: //u
        printch('u');
        buffstr[i] = 'u';
        i++;
        break;
case 23: //i
        printch('i');
        buffstr[i] = 'i';
        i++;
        break;
```



```

case 24: //o
    printch('o');
    buffstr[i] = 'o';
    i++;
    break;
case 25: //p
    printch('p');
    buffstr[i] = 'p';
    i++;
    break;
case 26: // [
    printch((char)91); // Código ASCII para '['
    buffstr[i] = '[';
    i++;
    break;
case 27: // ]
    printch((char)93); // Código ASCII para ']'
    buffstr[i] = ']';
    i++;
    break;
case 28: //ENTER
    printch('\n'); // Nova linha
    buffstr[i] = '\n';
    i++;
    break;
case 29: //CTRL
    //0x80 = usada para verificar se uma tecla foi liberada.
    if (inportb(0x60) & 0x80) {
        // Se a tecla 'Ctrl' foi liberada
        print("Ctrl Released");
    } else {
        // Se a tecla 'Ctrl' foi pressionada
        print("Ctrl Pressed");
    }
    break;
case 30: //a
    printch('a');
    buffstr[i] = 'a';
    i++;
    break;
case 31: //s
    printch('s');
    buffstr[i] = 's';
    i++;
    break;
case 32: //d
    printch('d');
    buffstr[i] = 'd';
    i++;
    break;
case 33: //f
    printch('f');
    buffstr[i] = 'f';
    i++;
    break;
case 34: //g
    printch('g');
    buffstr[i] = 'g';
    i++;
    break;

```

```
case 35: //h
    printch('h');
    buffstr[i] = 'h';
    i++;
    break;
case 36: //j
    printch('j');
    buffstr[i] = 'j';
    i++;
    break;
case 37: //k
    printch('k');
    buffstr[i] = 'k';
    i++;
    break;
case 38: //l
    printch('l');
    buffstr[i] = 'l';
    i++;
    break;
case 39: //;
    printch(';');
    buffstr[i] = ';';
    i++;
    break;
case 40: // '
    printch("\'");
    buffstr[i] = "\'";
    i++;
    break;
case 41: // `
    printch("`");
    buffstr[i] = "`";
    i++;
    break;
case 42: //LSHIFT
    if (inportb(0x60) & 0x80) {
        // Se a tecla 'Shift' foi liberada
        print("LShift Released");
    } else {
        // Se a tecla 'Shift' foi pressionada
        print("LShift Pressed");
    }
    break;
case 43: // \
    printch("\\");
    buffstr[i] = "\\";
    i++;
    break;
case 44: //z
    printch('z');
    buffstr[i] = 'z';
    i++;
    break;
case 45: //x
    printch('x');
    buffstr[i] = 'x';
    i++;
    break;
case 46: //c
```

```

        printch('c');
        buffstr[i] = 'c';
        i++;
        break;
case 47: //v
        printch('v');
        buffstr[i] = 'v';
        i++;
        break;
case 48: //b
        printch('b');
        buffstr[i] = 'b';
        i++;
        break;
case 49: //n
        printch('n');
        buffstr[i] = 'n';
        i++;
        break;
case 50: //m
        printch('m');
        buffstr[i] = 'm';
        i++;
        break;
case 51: // ,
        printch(',');
        buffstr[i] = ',';
        i++;
        break;
case 52: // .
        printch('.');
        buffstr[i] = '.';
        i++;
        break;
case 53: // /
        printch('/');
        buffstr[i] = '/';
        i++;
        break;
case 54: // RShift
        if (inportb(0x60) & 0x80) {
            // Se a tecla 'RShift' foi liberada
            print("RShift Released");
        } else {
            // Se a tecla 'RShift' foi pressionada
            print("RShift Pressed");
        }
        break;
case 55: // PRINTSCREEN - PrtSc
        // Tecla 'PrtSc'
        print("PrtSc");
        break;
case 56: // ALT
        if (inportb(0x60) & 0x80) {
            // Se a tecla 'Alt' foi liberada
            print("Alt Released");
        } else {
            // Se a tecla 'Alt' foi pressionada
            print("Alt Pressed");
        }

```

```

        break;
    case 57: // SPACE
        // Tecla 'Space'
        printch(' '); // Adiciona um espaço
        buffstr[i] = ' ';
        i++;
        break;
    }
}
buffstr[i] = 0;
return buffstr;
}

```

Código 8 – Conteúdo completo/atualizado do arquivo kb.c.

1.3. Testando o nosso shell simples

Agora para conseguirmos testá-lo, atualizaremos o arquivo kernel.c (código 9) e atualizaremos nosso script build.sh (código 10).

```

#include "basic/include/screen.h"
void kmain() {
    clearScreen();
    print(": : Digite algo:");

    // Chama a função readStr para obter a entrada do usuário
    updateCursor();
    string userInput = readStr();

    // Limpa a tela e imprime a string lida
    clearScreen();
    print(": : Voce digitou:");
    print(userInput);
    print("\n\napos ter pressionado ESC, o programa foi encerrado...");
}

```

Código 9 – Atualização do arquivo kernel.c.

```

#!/bin/sh
nasm -f elf32 kernel.asm -o kasm.o
gcc -m32 -c -w kernel.c -o kc.o -ffreestanding
gcc -m32 -c -w screen.c -o screen.o -ffreestanding
gcc -m32 -c -w string.c -o string.o -ffreestanding
gcc -m32 -c -w system.c -o system.o -ffreestanding
gcc -m32 -c -w kb.c -o kb.o -ffreestanding
ld -m elf_i386 -T link.ld -o basic/boot/kernel.bin kasm.o kc.o screen.o string.o system.o kb.o

```

Código 10 – Atualização do script build.sh.

```
vboxuser@Debian12:~/Projeto_2/so$ tree
.
├── basic
│   ├── boot
│   │   ├── grub
│   │   │   └── grub.cfg
│   │   └── kernel.bin
│   └── include
│       ├── kb.h
│       ├── screen.h
│       ├── string.h
│       ├── system.h
│       └── types.h
├── basic.iso
├── build.sh
├── kasm.o
├── kb.c
├── kb.o
├── kc.o
├── kernel.asm
├── kernel.c
├── kernel.o
├── link.ld
├── make.sh
├── run.sh
├── screen.c
├── screen.o
├── string.c
├── string.o
├── system.c
└── system.o

5 directories, 25 files
```

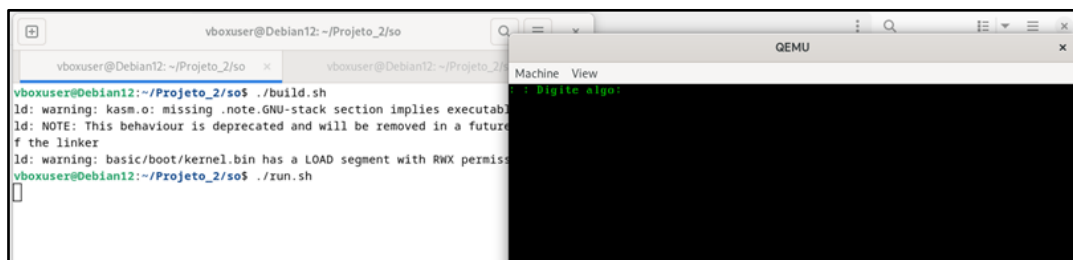
Figura 3 – Visão geral da estrutura de arquivos. Criar pasta “so”.

Esse código do kernel.c vai fazer o seguinte, ele vai pedir ao usuário digitar algo, sendo assim, o usuário pode digitar qualquer coisa porém existem algumas limitações, tais como:

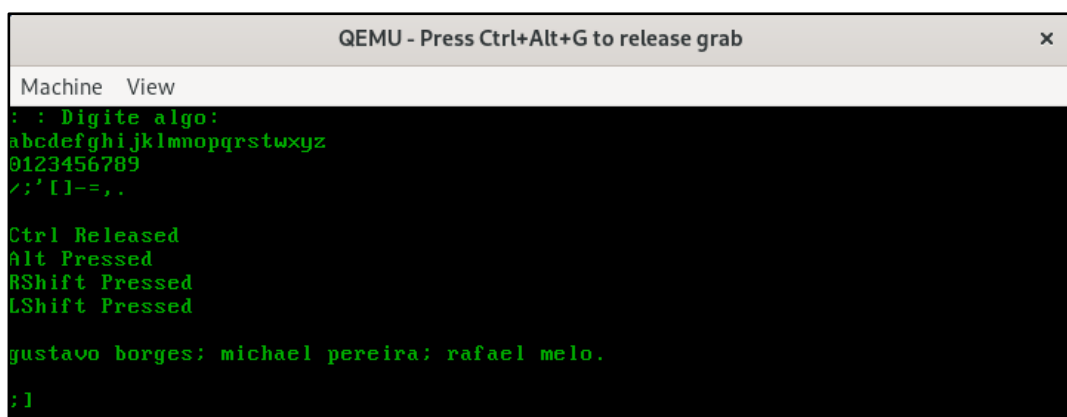
- Não foi implementado perfeitamente o uso de ALT, LSHIFT, RSHIFT, CTRL e PrtSc;
- Como não foi implementado o uso das teclas anteriormente ditas, não será possível usar combinações tais como ?, !, @, #, %, ^, & e etc.
- A tecla “\” funciona porém existe alguma barreira no QEMU que impede de mostrar esse símbolo no vídeo para nós. Tentamos usar a sintaxe printf("\n"), porém o símbolo não é mostrado na input mas na output (ao pressionar ESC) funciona perfeitamente (o símbolo é visualizado).

Se pressionarmos ESC, o programa é finalizado, mas antes ele vai limpar toda a tela e retornar todo o conjunto de strings e parâmetros utilizados para o usuário verificar, vale ressaltar que, ao usar CTRL, Lshift, Rshift, Alt é mostrado uma mensagem de que a tecla foi pressionada, porém no output não será mostrado nada pois decidimos fazer absolutamente com esse valor no momento.

Então vamos lá compilar nosso código conforme a Figura 4.



Abaixo fizemos alguns testes para o entendimento do mesmo, sendo assim, começaremos com o alfabeto, utilizaremos a tecla ENTER para saltar uma linha, iniciaremos a linha com números de 0-9, saltaremos outra linha, e iniciaremos uma linha com símbolos em geral e por fim, saltaremos várias linhas porém cada linha eu utilizei uma tecla de atalho diferente para mostrar que a tecla foi pressionada de fato (Figura 5).



Infelizmente não foi muito bom este teste que realizamos, então vamos fazer por partes. Sendo assim, dividiremos cada linha como um único teste conforme veremos abaixo.

Começaremos com um teste digitando apenas caracteres do alfabeto de a-z e demonstraremos a output para irmos direto ao ponto.

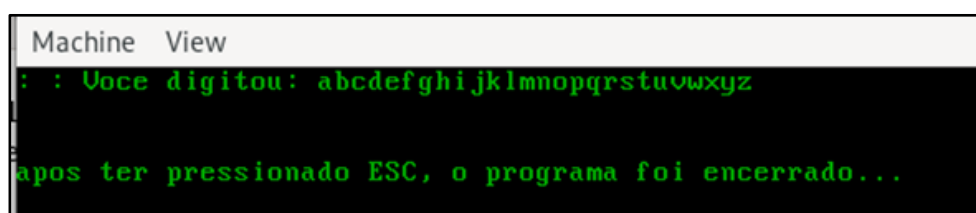
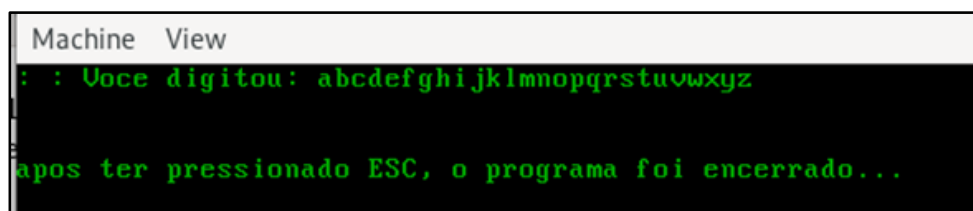


Figura 7 – Teste 2 – OUTPUT alfabeto.

Para cada teste, precisamos encerrar a nossa máquina QEMU pois nosso código encerra o programa e não existe outra interação a não ser fechar a janela e executar o script run.sh novamente.

Agora iremos fazer um teste digitando apenas números de 0-9.

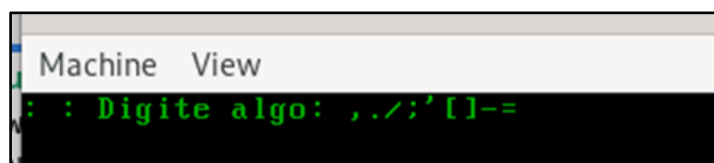


```
Machine View
: : Voce digitou: abcdefghijklmnopqrstuvwxyz
apos ter pressionado ESC, o programa foi encerrado...
```

Figura 8 – Teste 3 – OUTPUT números.

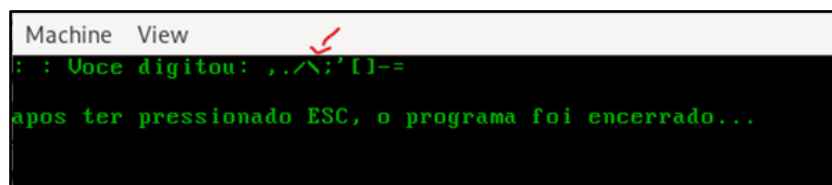
Agora iremos fazer um teste digitando apenas símbolos possíveis com a nossa configuração atual (temos combinações limitadas sem usar o CTRL, Rshift, Lshift e Alt).

Vale ressaltar que, nós havíamos dito que o uso de “\” por algum motivo não estava apresentando nos inputs, porém aqui já conseguimos observá-lo.



```
Machine View
: : Digite algo: ,./;\'[]-=
```

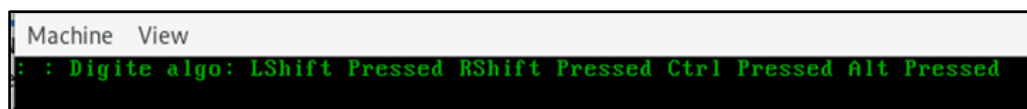
Figura 9 – Teste 4 – INPUT símbolos.



```
Machine View
: : Voce digitou: ,./;\'[]-=
apos ter pressionado ESC, o programa foi encerrado...
```

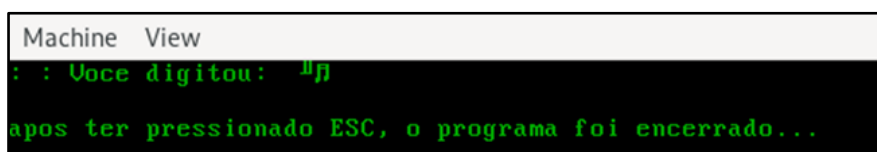
Figura 10 – Teste 4 – OUTPUT símbolos.

Agora testamos os atalhos CTRL, Rshfit, Lshift e Alt.



```
Machine View
: : Digite algo: LShift Pressed RShift Pressed Ctrl Pressed Alt Pressed
```

Figura 11 – Teste 5 – INPUT atalhos.



```
Machine View
: : Voce digitou:  apos ter pressionado ESC, o programa foi encerrado...
```

Figura 12 – Teste 5 – OUTPUT atalhos.

Tentaremos agora o uso de TAB e ENTER.

```
Machine View
: : Digite algo:      tab      tab      tab comando usado foi t
esse foi enter n
```

Figura 13 – Teste 6 – INPUT atalhos.

```
Machine View
: : Voce digitou:      tab      tab      tab comando usado foi \t
esse foi enter \n

apos ter pressionado ESC, o programa foi encerrado...
```

Figura 14 – Teste 6 – OUTPUT atalhos.

O teste foi finalizado, agora precisaremos fazer um arquivo ISO do nosso projeto. Porém para não ficar muito carregado a nossa tarefa, eu optei por pular as figuras, pois já foi explicado o método utilizado para fazermos o nosso arquivo ISO nas tarefas 1 e 2.

Em resumo, utilizaremos nosso script make.sh e iremos gerar uma ISO, esse arquivo iremos enviar para o nosso host através do protocolo SCP (Secure Copy Protocol). Com a ISO em mãos, mudaremos em nossas configurações da máquina virtual chamada BasicOS o seu disco óptico (“utilizaremos outro CD”), e após trocarmos para nossa “.iso”, estaremos com nosso sistema executando sem problemas conforme a Figura 15.

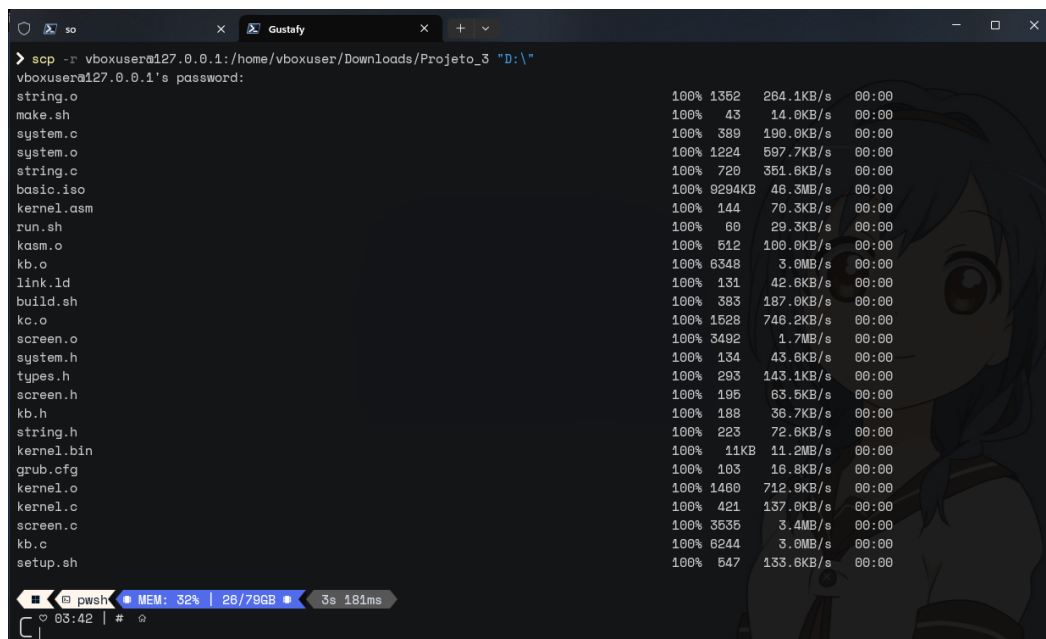


Figura 15 – Copiando arquivos entre guest e hospedeiro.

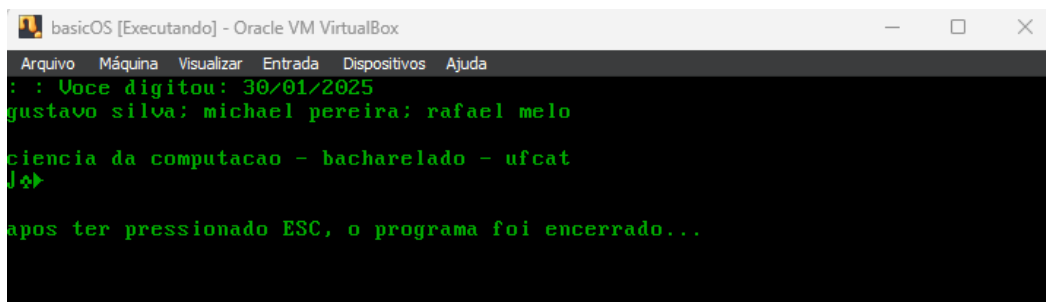


Figura 16 – Testando nossa ISO em nossa Máquina Virtual.

Um adendo importante, é que enquanto o relatório estava sendo desenvolvido, alteramos o nome da pasta raiz de “Projeto_2” para “Projeto_3”.

2. Tarefa 2: implementação de interrupções

Os sinais de interrupção servem para informar a CPU sobre eventos externos e é basicamente isso que iremos implementar nessa tarefa, sendo assim, primeiramente iremos informar ao processador sobre os processos. Para isso, iremos criar um vetor de 256 elementos, cada um contendo a informação das interrupções a serem implementadas. O local exato de criação desse vetor não é crucial, mas é essencial que o computador saiba onde ele está na memória. Após definir o vetor, é necessário informar ao processador onde essa variável será armazenada. Além disso, o tamanho do vetor na memória é comunicado ao computador por meio da instrução assembly "lidt" (**Load Interrupt Description Table**). A Interrupt Description Table (IDT) é, essencialmente, o vetor mencionado anteriormente. Em resumo, o processo envolve:

- 1) Criação do vetor;
- 2) Definição de cada elemento do vetor;
- 3) Salvamento do endereço do primeiro elemento e seu tamanho em outro objeto, que é enviado para a CPU através da instrução assembly "lidt".

Nesta tarefa, as alterações no sistema operacional (SO) não serão visíveis, pois as funções serão implementadas na estrutura interna do sistema. Serão criados então, os arquivos **util.h**, **util.c**, **idt.h**, **idt.c**, **isr.h** e **isr.c**. Vale ressaltar que arquivos de cabeçalhos foram salvos dentro da pasta **include** para melhor organização e o resto do código e posteriormente binários serão salvos dentro da pasta do S.O. conforme na Figura 17.

```
vboxuser@Debian12:~/Projeto_3/so/basic/include$ touch util.h idt.h isr.h
vboxuser@Debian12:~/Projeto_3/so/basic/include$ ls
idt.h isr.h kb.h screen.h string.h system.h types.h util.h
vboxuser@Debian12:~/Projeto_3/so/basic/include$ cd
vboxuser@Debian12:~$ cd Projeto_3/so
vboxuser@Debian12:~/Projeto_3/so$ touch util.c idt.c isr.c
vboxuser@Debian12:~/Projeto_3/so$ ls
basic      idt.c  kb.c  kernel.asm  link.ld  screen.c  string.o  util.c
basic.iso  isr.c  kb.o  kernel.c    make.sh  screen.o  system.c
build.sh   kasm.o kc.o  kernel.o    run.sh   string.c  system.o
vboxuser@Debian12:~/Projeto_3/so$ tree
```

Figura 17 – Criação dos arquivos e organização.

Antes de abordarmos o conteúdo dos arquivos criados, é importante dizer que duas novas funções inline serão adicionadas ao arquivo `types.h` conforme o código 11 abaixo.

```
#ifndef TYPES_H
#define TYPES_H

typedef signed char int8;
typedef unsigned char uint8;

typedef signed short int16;
typedef unsigned short uint16;

typedef signed int int32;
typedef unsigned int uint32;

typedef signed long int64;
typedef unsigned long uint64;

typedef char* string;

#define low_16(address) (uint16)((address) & 0xFFFF)
#define high_16(address) (uint16)(((address) >> 16) & 0xFFFF)

#endif
```

Código 11 – Atualização do código `types.h`.

Em relação a estas funções, as funções `low_16()` e `high_16()` são definidas para operar em inteiros de 32 bits. A função `low_16()` extrai os 16 bits menos significativos (bits de ordem mais baixa) de um inteiro de 32 bits, enquanto a função `high_16()` extrai os 16 bits mais significativos (bits de ordem mais alta) do mesmo inteiro. Essas funções são úteis para manipular e isolar partes específicas de endereços de 32 bits, fornecendo flexibilidade no trabalho com dados de tamanho maior.

Agora iremos falar especificamente dos arquivos `util.h` e `util.c`. Cujo, no arquivo `util.h` (Código 12) são criadas três funções: `memory_copy()`, `memory_set` e `int_to_ascii()`.

```
#ifndef UTIL_H
#define UTIL_H

#include "types.h"

void memory_copy(char *source, char *dest, int nbytes);
void memory_set(uint8 *dest, uint8 val, uint32 len);
void int_to_ascii(int n, char str[]);

#endif
```

Código 12 – Conteúdo do código `util.h`.

A função `memory_copy()` copia os `nbytes` caracteres da origem para o destino. `memory_set()` atribui um valor específico “val” para o destino, repetindo esse valor nas “len” posições do vetor. Por fim, `int_to_ascii()` converte um valor inteiro em uma string.

Agora iremos realizar a implementação completa dos métodos apresentados no cabeçalho para o arquivo `util.c` (Código 13) no qual os mesmos já foram explicados

anteriormente, e para facilitar a compreensão adicionamos comentários no código.

```
#include "basic/include/util.h"

// : : Implementação da função memory_copy : :
void memory_copy(char *source, char *dest, int nbytes) {
    for (int i = 0; i < nbytes; i++) {
        dest[i] = source[i];
    }
}

// : : Implementação da função memory_set : :
void memory_set(uint8 *dest, uint8 val, uint32 len) {
    for (uint32 i = 0; i < len; i++) {
        dest[i] = val;
    }
}

// : : Implementação da função int_to_ascii : :
void int_to_ascii(int n, char str[]) {
    int i = 0, sign = 1;
    // Lida com números negativos
    if (n < 0) {
        sign = -1;
        n = -n;
    }
    // Converte cada dígito para ASCII e armazena na string
    do {
        str[i++] = n % 10 + '0';
    } while ((n /= 10) > 0);

    // Adiciona sinal negativo se necessário
    if (sign == -1) {
        str[i++] = '-';
    }
    // Termina a string
    str[i] = '\0';

    // Inverte a string
    int start = 0;
    int end = i - 1;
    while (start < end) {
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}
```

Código 13 – Conteúdo do código util.c.

Agora que falamos dos códigos util.h e util.c, iremos especificar idt.h e idt.c. Sendo assim, em relação ao código de cabeçalho idt.h KERNEL_CS (Código 14), é o seletor de segmento referente ao segmento do kernel. Além disso, são definidas as estruturas idt_gate_t e idt_register_t.

```
#ifndef IDT_H
#define IDT_H
```

```

#include "types.h"
#define KERNEL_CS 0X08

typedef struct {
    uint16 low_offset;
    uint16 sel;
    uint8 always0;
    uint8 flags;
    uint16 high_offset;
} __attribute__((packed)) idt_gate_t;

typedef struct {
    uint16 limit;
    uint32 base;
} __attribute__((packed)) idt_register_t;

#define IDT_ENTRIES 256
idt_gate_t idt[IDT_ENTRIES];
idt_register_t idt_reg;

/* Funções implementadas em idt.c */
void set_idt_gate(int n, uint32 handler);
void set_idt();
#endif

```

Código 14 – Conteúdo do código idt.h.

Vale ressaltar que, a primeira estrutura descreve cada elemento da Interrupt Descriptor Table (IDT), que contém 256 elementos. Cada byte desses elementos tem um significado, e a estrutura é mapeada para evitar a necessidade de usar instruções de baixo nível para acessar individualmente esses bytes. A soma de todos os elementos na estrutura resulta em 64 bits, e o atributo `__attribute__((packed))` é adicionado para garantir que o compilador não adicione espaço ou código adicional, instruindo-o a gerar o espaço exatamente conforme aparece no código. Após criar essa estrutura, é criado um vetor de 256 elementos.

Além disso, a segunda estrutura, `idt_register_t`, é projetada para representar o registrador da IDT (Interrupt Descriptor Table). Utilizando o atributo `__attribute__((packed))`, ela evita otimizações e modificações indesejadas do compilador. Com um tamanho total de exatos 48 bits, seus primeiros 16 bits indicam o "limit", e os últimos 32 bits especificam a "base". O campo "base" contém o endereço do vetor `idt[IDT_ENTRIES]` (o início do vetor), enquanto o "limit" armazena o tamanho de cada elemento na IDT. Dado que esses valores são constantes para todos os elementos do vetor, apenas um único elemento chamado `idt_reg` é criado para representar o registrador da IDT. Essa estrutura é essencial para configurar e informar ao processador a localização e tamanho da IDT.

Por outro lado, em relação ao `idt.c` (Código 15), podemos observar a implementação das funções presentes em `idt.h` (Código 14) tais como `set_idt_gate()` e `set_idt()`.

```

#include "basic/include/idt.h"
#include "basic/include/util.h"

```

```

void set_idt_gate(int n, uint32 handler) {
    idt[n].low_offset = low_16(handler);
    idt[n].sel = KERNEL_CS;
    idt[n].always0 = 0;
    idt[n].flags = 0x8E;
    idt[n].high_offset = high_16(handler);
}

void set_idt() {
    idt_reg.base = (uint32) &idt;
    idt_reg.limit = IDT_ENTRIES * sizeof(idt_gate_t) - 1;
    __asm__ __volatile__ ("lidt (%0)" : : "r" (&idt_reg));
}

```

Código 15 – Conteúdo do código idt.c.

A função `set_idt_gate()` preenche os campos da estrutura `idt_gate_t` para um gate específico na IDT, configurando-o para lidar com uma interrupção ou exceção, cujo a primeira propriedade, `low_offset`, é definida a partir da função `low_16()`, enquanto `high_offset` é obtida da função `high_16()`. Nota-se que `low_offset` é o primeiro espaço na estrutura, e `high_offset` é o segundo. A ordem desses elementos é crucial. A variável `sel` recebe o Kernel Segment Selector (valor 0x08), enquanto `always0` se refere ao "nível de execução" (ring 0 para o kernel), indicando se uma interrupção pode ser executada apenas no kernel, em espaço de usuário ou em um espaço intermediário. O valor de `flags` é constante e será 0x8E por padrão.

E na função `set_idt()`, é configurada a IDT como um todo. As ações realizadas incluem que `idt_reg.base` recebe o endereço do vetor `idt`, `idt_reg.limit` é calculado como o tamanho total da IDT em bytes (número de entradas multiplicado pelo tamanho de cada entrada) menos 1.

Além disso, a instrução `lidt` é utilizada via inline assembly para carregar o registrador IDTR (`idt_reg`) com os valores calculados. Isso informa ao processador onde a IDT está localizada na memória e seu tamanho.

Agora que temos as funções `set_idt_gate()` e `set_idt()`, iremos implementá-las nos códigos (Códigos 16 e 17) que são essenciais para configurar a IDT (Interrupt Descriptor Table) em um sistema operacional. A função `set_idt_gate()` é chamada várias vezes para ajustar os valores de elementos individuais no vetor `idt[IDT_ENTRIES]`, sendo necessário fornecer o número do elemento (`n`) e o handler (um inteiro de 32 bits) que define o conteúdo da estrutura. Já a função `set_idt()` é chamada uma única vez para configurar a IDT como um todo, passando o vetor resultante `idt[IDT_ENTRIES]` para o processador. O valor de base é o endereço base do vetor (`&idt`), e `limit` recebe o tamanho do vetor. Após o preenchimento de base e `limit`, a instrução assembly `lidt` é chamada, passando o endereço de `idt_reg` (`&idt_reg`), permitindo que o processador lide com a IDT.

O próximo passo envolve a especificação da Interrupt Service Routine (Rotina de Serviço de Interrupção ou ISR), que é crucial para lidar com eventos como exceções (por exemplo, divisão por zero). A implementação completa de `isr.h` está presente no Código 17.

```

#ifndef ISR_H
#define ISR_H

#include "types.h"

/* ISRs reservadas para exceções de CPU */
void isr0();
void isr1();
void isr2();
void isr3();
void isr4();
void isr5();
void isr6();
void isr7();
void isr8();
void isr9();
void isr10();
void isr11();
void isr12();
void isr13();
void isr14();
void isr15();
void isr16();
void isr17();
void isr18();
void isr19();
void isr20();
void isr21();
void isr22();
void isr23();
void isr24();
void isr25();
void isr26();
void isr27();
void isr28();
void isr29();
void isr30();
void isr31();

extern string exception_messages[32];

void isr_install();

#endif

```

Código 16 – Conteúdo do código isr.h.

No Código 17, a função `isr_install()` é implementada para preencher o vetor `idt[IDT_ENTRIES]` com os endereços das funções de manipulação que devem ser chamadas quando ocorre uma interrupção. Cada função ISR (Interrupt Service Routine) é associada a um número específico de interrupções. Por exemplo, a divisão por zero está associada à `isr0()`, a interrupção 1 à `isr1()`, e assim por diante. Essas funções estão definidas no mesmo arquivo. Quando uma interrupção ocorre, a função correspondente é chamada, uma mensagem de exceção é impressa, e a execução da CPU é interrompida usando a instrução assembly `"hlt"`.

O vetor de mensagens de exceção também é definido no arquivo `isr.c` (Código 17), contendo 32 mensagens correspondentes às 32 ISRs implementadas. A partir desse ponto,

o S.O. está habilitado para lidar com vários tipos de interrupções, como exceções, cliques de teclado, movimentos de mouse, pulsos do relógio, entre outros.

```
#include "basic/include/isr.h"
#include "basic/include/idt.h"
#include "basic/include/util.h"

void isr_install() {
    set_idt_gate(0, (uint32)isr0);
    set_idt_gate(1, (uint32)isr1);
    set_idt_gate(2, (uint32)isr2);
    set_idt_gate(3, (uint32)isr3);
    set_idt_gate(4, (uint32)isr4);
    set_idt_gate(5, (uint32)isr5);
    set_idt_gate(6, (uint32)isr6);
    set_idt_gate(7, (uint32)isr7);
    set_idt_gate(8, (uint32)isr8);
    set_idt_gate(9, (uint32)isr9);
    set_idt_gate(10, (uint32)isr10);
    set_idt_gate(11, (uint32)isr11);
    set_idt_gate(12, (uint32)isr12);
    set_idt_gate(13, (uint32)isr13);
    set_idt_gate(14, (uint32)isr14);
    set_idt_gate(15, (uint32)isr15);
    set_idt_gate(16, (uint32)isr16);
    set_idt_gate(17, (uint32)isr17);
    set_idt_gate(18, (uint32)isr18);
    set_idt_gate(19, (uint32)isr19);
    set_idt_gate(20, (uint32)isr20);
    set_idt_gate(21, (uint32)isr21);
    set_idt_gate(22, (uint32)isr22);
    set_idt_gate(23, (uint32)isr23);
    set_idt_gate(24, (uint32)isr24);
    set_idt_gate(25, (uint32)isr25);
    set_idt_gate(26, (uint32)isr26);
    set_idt_gate(27, (uint32)isr27);
    set_idt_gate(28, (uint32)isr28);
    set_idt_gate(29, (uint32)isr29);
    set_idt_gate(30, (uint32)isr30);
    set_idt_gate(31, (uint32)isr31);
    set_idt(); // Carregado com ASM
}

/* Handlers */
void isr0() {
    print(exception_messages[0]);
    asm("hlt");
}

void isr1() {
    print(exception_messages[1]);
    asm("hlt");
}

void isr2() {
    print(exception_messages[2]);
    asm("hlt");
}
```

```
void isr3() {
    print(exception_messages[3]);
    asm("hlt");
}

void isr4() {
    print(exception_messages[4]);
    asm("hlt");
}

void isr5() {
    print(exception_messages[5]);
    asm("hlt");
}

void isr6() {
    print(exception_messages[6]);
    asm("hlt");
}

void isr7() {
    print(exception_messages[7]);
    asm("hlt");
}

void isr8() {
    print(exception_messages[8]);
    asm("hlt");
}

void isr9() {
    print(exception_messages[9]);
    asm("hlt");
}

void isr10() {
    print(exception_messages[10]);
    asm("hlt");
}

void isr11() {
    print(exception_messages[11]);
    asm("hlt");
}

void isr12() {
    print(exception_messages[12]);
    asm("hlt");
}

void isr13() {
    print(exception_messages[13]);
    asm("hlt");
}

void isr14() {
    print(exception_messages[14]);
    asm("hlt");
}
```



```
void isr15() {
    print(exception_messages[15]);
    asm("hlt");
}

void isr16() {
    print(exception_messages[16]);
    asm("hlt");
}

void isr17() {
    print(exception_messages[17]);
    asm("hlt");
}

void isr18() {
    print(exception_messages[18]);
    asm("hlt");
}

void isr19() {
    print(exception_messages[19]);
    asm("hlt");
}

void isr20() {
    print(exception_messages[20]);
    asm("hlt");
}

void isr21() {
    print(exception_messages[21]);
    asm("hlt");
}

void isr22() {
    print(exception_messages[22]);
    asm("hlt");
}

void isr23() {
    print(exception_messages[23]);
    asm("hlt");
}

void isr24() {
    print(exception_messages[24]);
    asm("hlt");
}

void isr25() {
    print(exception_messages[25]);
    asm("hlt");
}

void isr26() {
    print(exception_messages[26]);
    asm("hlt");
}
```

[illegible]

```
"Reserved",  
// Continue preenchendo conforme necessário  
};
```

Código 17 – Conteúdo do código isr.c.

Agora que finalizamos a implementação dos nossos códigos, antes de testarmos, precisamos pelo menos verificar se a nossa build estará correta, sendo assim, realizamos a seguinte modificação do arquivo build.sh conforme o Código 18.

```
#!/bin/sh  
nasm -f elf32 kernel.asm -o kasm.o  
gcc -m32 -c -w kernel.c -o kc.o -ffreestanding  
gcc -m32 -c -w screen.c -o screen.o -ffreestanding  
gcc -m32 -c -w string.c -o string.o -ffreestanding  
gcc -m32 -c -w system.c -o system.o -ffreestanding  
gcc -m32 -c -w kb.c -o kb.o -ffreestanding  
gcc -m32 -c -w util.c -o util.o -ffreestanding  
gcc -m32 -c -w idt.c -o idt.o -ffreestanding  
gcc -m32 -c -w isr.c -o isr.o -ffreestanding  
ld -m elf_i386 -T link.ld -o basic/boot/kernel.bin kasm.o kc.o screen.o string.o system.o kb.o util.o idt.o isr.o
```

Código 18 – Atualização do código build.sh.

Mas infelizmente tivemos erros com a compilação, sendo assim, foram realizadas algumas modificações que iremos explicar brevemente.

Nos arquivos de cabeçalho, o idt.h, eu deixei as variáveis como “extern” (Código 19).

```
#ifndef IDT_H  
#define IDT_H  
#include "types.h"  
#define KERNEL_CS 0X08  
  
typedef struct {  
    uint16 low_offset;  
    uint16 sel;  
    uint8 always0;  
    uint8 flags;  
    uint16 high_offset;  
} __attribute__((packed)) idt_gate_t;  
  
typedef struct {  
    uint16 limit;  
    uint32 base;  
} __attribute__((packed)) idt_register_t;  
  
#define IDT_ENTRIES 256  
extern idt_gate_t idt[IDT_ENTRIES];  
extern idt_register_t idt_reg;  
  
/* Funções implementadas em idt.c */  
void set_idt_gate(int n, uint32 handler);
```

```
void set_idt();  
#endif
```

Código 19 – Atualização do código idt.h.

No arquivo idt.c, foi declarado duas variáveis de forma global, para que possamos criar uma única instância das variáveis idt e idt_reg que podem ser acessadas em outros arquivos fonte (como isr.c, por exemplo) ao incluir o cabeçalho idt.h. Isso ajuda na modularização e na reutilização de código (Código 20).

```
#include "basic/include/idt.h"  
#include "basic/include/util.h"  
  
idt_gate_t idt[IDT_ENTRIES];  
idt_register_t idt_reg;  
  
void set_idt_gate(int n, uint32 handler) {  
    idt[n].low_offset = low_16(handler);  
    idt[n].sel = KERNEL_CS;  
    idt[n].always0 = 0;  
    idt[n].flags = 0x8E;  
    idt[n].high_offset = high_16(handler);  
}  
  
void set_idt() {  
    idt_reg.base = (uint32) &idt;  
    idt_reg.limit = IDT_ENTRIES * sizeof(idt_gate_t) - 1;  
    __asm__ __volatile__ ("lidt (%0)" : : "r" (&idt_reg));  
}
```

Código 19 – Atualização do código idt.c.

Após isto, a build funcionou corretamente conforme a Figura 18.

```
vboxuser@Debian12:~/Projeto_3/so$ ./build.sh  
ld: warning: kasm.o: missing .note.GNU-stack section implies executable stack  
ld: NOTE: This behaviour is deprecated and will be removed in a future version of  
the linker  
ld: warning: basic/boot/kernel.bin has a LOAD segment with RWX permissions
```

Figura 18 – Resultado da compilação.

Agora iremos prosseguir com a atividade, precisamos de alguma forma testarmos as nossas interrupções, e para isso, optamos por utilizar uma chamada de interrupção no código kernel.c (Código 20). Quando esse evento específico ocorrer (por exemplo, uma tecla específica pressionada), isso nos permitirá incluir uma chamada de interrupção diretamente no código. Isso pode ser feito usando uma instrução assembly no código C. Por exemplo, se a interrupção que desejamos testar seja a ISR19, poderíamos simplesmente adicionar algo assim em nosso código kernel.c: asm("int \$19") ou isr19().

Sendo assim, abaixo mostro os seguintes trechos modificados nos códigos para adaptação do nosso teste final.

No arquivo kb.c optamos por deixar de usar a tecla ESC para finalizar o programa e optamos por usar a tecla ENTER para prosseguir para próxima janela, e ESC para limpar

a tela e também para acionar a interrupção conforme o trecho parcial do kb.c abaixo (código 20).

```
#include "basic/include/kb.h"
#include "basic/include/idt.h"

string readStr() {
    ...

    switch(inportb(0x60)){
        case 1: //ESC
            clearScreen(); // Limpa a tela
            i = 0; // Reinicia o índice da string
            break;
        ...

    case 28: //ENTER
            reading = 0; // Encerra a leitura/loop
            break;

    ...
    }
}
```

Código 20 – Atualização do código kb.c.

De foco, pensamos em usar a interrupção 19 ou 20 conforme a Figura 19 abaixo.

```
203
204 /* Para imprimir as mensagens que definem cada exceção */
205 string exception_messages[] = {
206     "Division by Zero",
207     "Debug",
208     "Non Maskable Interrupt",
209     "Breakpoint",
210     "Into Detected Overflow",
211     "Out of Bounds",
212     "Invalid Opcode",
213     "No Coprocessor",
214     "Double Fault",
215     "Coprocessor Segment Overrun",
216     "Bad TSS",
217     "Segment Not Present",
218     "Stack Fault",
219     "General Protection Fault",
220     "Page Fault",
221     "Unknown Interrupt",
222     "Coprocessor Fault",
223     "Alignment Check",
224     "Machine Check",
225     "\nSESSÃO FINALIZADA - Processo killed",
226     "Interrupcao arbitaria",
227     "Reserved",
228     "Reserved"
```

Figura 19 – Resultado da compilação.

O kernel.c ficou da seguinte forma para teste conforme o código 21 abaixo.

```
#include "basic/include/idt.h"
#include "basic/include/isr.h"
#include "basic/include/screen.h"

void keyboard_handler() {
    // aqui sera lidado com a interrupção do teclado
    // ler o scancode do teclado por exemplo
}

void kmain() {
    // Configuração inicial

    clearScreen();
    print(": : Digite algo:");

    // Configura o handler da interrupção do teclado (0x21)
    set_idt_gate(33, (uint32)keyboard_handler);
    set_idt();
    // Chama a função readStr para obter a entrada do usuário
    updateCursor();
    string userInput = readStr();

    // Limpa a tela e imprime a string lida
    clearScreen();
    print(": : Voce digitou:");
    print(userInput);
    print("\n\nEsperando interrupção do teclado...");

    // Não é necessário chamar asm("int $0x19");
    // O código abaixo só será executado após a interrupção do teclado
    clearScreen();
    print(": : Introducao da interrupcao do teclado!");
}
```

Código 21 – Atualização do código kernel.c.

Resultado final.

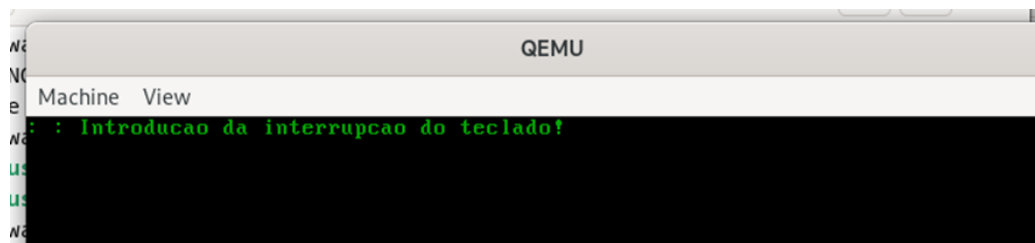


Figura 20 – Resultado da compilação.

2.1. Correções realizadas no dia 02/02/2025

Antes de iniciarmos a tarefa 3, gostaríamos de ressaltar que para a próxima tarefa iremos modificar a estrutura do nosso projeto para figura 21.

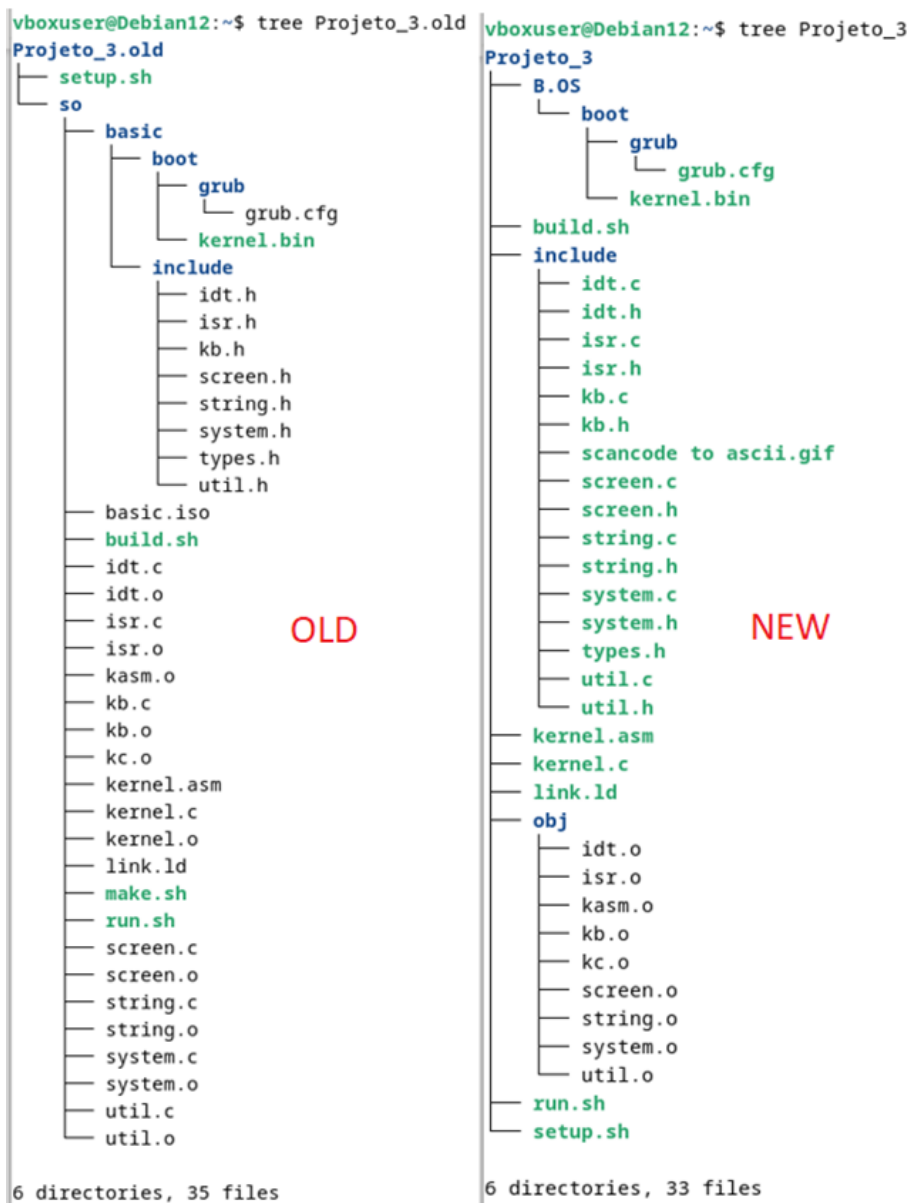


Figura 21 – Correções e atualização da estrutura do projeto.

O que fizemos? Basicamente reestruturamos a estrutura do nosso projeto, deixamos do lado de fora os scripts, makefile e .iso, e criamos 3 pastas “[B.OS],[obj] e [include]”, no qual dentro de B.OS ficou apenas arquivos de boot do kernel e grub, dentro de include os arquivos de cabeçalho, implementações e linkagem.

Conquanto, deixamos preparado para a próxima tarefa na qual iremos modificar a estrutura do projeto e seu mecanismo de construção (build) para podermos utilizar o make.

3. Tarefa 3: modificação da estrutura do projeto e build

Antes de iniciarmos a tarefa 3, precisamos ressaltar que aplicamos todas as correções realizadas pelo professor e sugestões do vídeo da tarefa 2 disponibilizadas via google classroom para que possamos continuar com as próximas tarefas.

Além disso, conforme havíamos dito no tópico 2.1, ficou pendente a questão de organizarmos a estrutura de arquivos do projeto, portanto, vamos primeiramente modificar a estrutura do projeto e seu mecanismo de construção (build) para podermos utilizar o make.

Sendo assim, na raiz do nosso projeto, onde está localizado o nosso script build.sh, vamos criar um arquivo Makefile, o qual ficará responsável (em consonância com a ferramenta make) por gerenciar o processo de compilação e geração do kernel do SO.

Vale ressaltar que, make é uma ferramenta que permite a construção de um projeto de forma bastante “inteligente”. Isto é, refere-se ao fato de que, no caso do uso do make, se compilamos alguns arquivos e porventura, atualizar um deles, apenas este será recompilado.

Para prosseguirmos, iremos falar em partes: **estrutura do projeto, makefile, funcionalidades do makefile, testando na prática.**

3.1. Estrutura do projeto

Conforme havíamos mostrado anteriormente na Figura 21 (Tarefa 2), aplicamos a correção para incorporar a utilização do make e manter uma organização mais eficiente dos arquivos. A árvore de diretórios e os principais arquivos fonte são os seguintes:

*inconsistente
cl o código fornecido.*

```
Projeto_3
├── B.OS
│   ├── boot
│   │   ├── grub
│   │   │   ├── grub.cfg
│   │   │   └── kernel.bin
│   ├── build.sh
│   ├── include
│   │   ├── idt.c
│   │   ├── idt.h
│   │   ├── isr.c
│   │   ├── isr.h
│   │   ├── kb.c
│   │   ├── kb.h
│   │   ├── scancode to ascii.gif
│   │   ├── screen.c
│   │   ├── screen.h
│   │   ├── string.c
│   │   ├── string.h
│   │   ├── system.c
│   │   ├── system.h
│   │   ├── types.h
│   │   └── util.c
│   ├── util.h
│   ├── kernel.asm
│   ├── kernel.c
│   ├── link.ld
│   └── obj
│       ├── idt.o
│       ├── isr.o
│       ├── kasm.o
│       ├── kb.o
│       ├── kc.o
│       ├── screen.o
│       ├── string.o
│       ├── system.o
│       └── util.o
├── run.sh
└── setup.sh
```

6 directories, 33 files

*Não deveriam estar
no src?*

Figura 22 – Estrutura do projeto atual.

*→ tem o
código
fornecido.*

- B.OS: Pasta principal do projeto que inclui códigos scripts, kernel.asm, kernel.c, link.ld e posteriormente o arquivo Makefile.
- boot: Contém arquivos relacionados ao processo de inicialização como o kernel.bin.
- grub: Armazena configurações do GRUB, o carregador de inicialização.
- obj: Diretório destinado a armazenar os arquivos objeto gerados durante a compilação.
- include: inclui os códigos-fonte e headers (cabeçalhos) que contém definições e declarações necessárias para a compilação dos códigos-fontes.

Essa estrutura é projetada para manter uma organização clara do projeto, facilitando a manutenção e compreensão.

3.2. Makefile

Nesta seção do relatório, descreveremos a implementação de um sistema operacional, destacando a utilização da ferramenta make para gerenciar eficientemente o processo de compilação e geração do kernel. A estrutura do projeto foi cuidadosamente organizada de acordo com as diretrizes fornecidas, culminando na criação do arquivo Makefile, que automatiza de maneira inteligente o fluxo de compilação.

O Makefile é uma peça fundamental, incorporando variáveis para armazenar informações cruciais, como compilador, linker, flags, entre outros. Criamos o arquivo usando editor nano (Figura 23) e sua estrutura é delineada a seguir no Código 22.



```
vboxuser@Debian12: ~/Projeto_3
vboxuser@Debian12:~/Projeto_3$ nano Makefile
vboxuser@Debian12:~/Projeto_3$
```

Figura 23 – Criando arquivo Makefile.

```
COMPILER = gcc
LINKER = ld
ASSEMBLER = nasm
CFLAGS = -m32 -c -ffreestanding
ASFLAGS = -f elf32
LDFLAGS = -m elf_i386 -T src/link.ld
EMULATOR = qemu-system-i386
EMULATOR_FLAGS = -kernel

SRCS = src/kernel.asm src/kernel.c src/idt.c src/isr.c src/kb.c src/screen.c
src/string.c src/system.c src/util.c
OBJS = obj/kasm.o obj/kc.o obj/idt.o obj/isr.o obj/kb.o obj/screen.o
obj/string.o obj/system.o obj/util.o
OUTPUT = B.OS/boot/kernel.bin

run: link
    $(EMULATOR) $(EMULATOR_FLAGS) $(OUTPUT)

link: compile $(OBJS)
    rm -rf B.OS
    mkdir -p B.OS/boot/grub
    echo set default=0 > B.OS/boot/grub/grub.cfg
    echo set timeout=0 >> B.OS/boot/grub/grub.cfg
    echo menuentry "B.OS" { >> B.OS/boot/grub/grub.cfg
    echo set root='(hd96)' >> B.OS/boot/grub/grub.cfg
    echo multiboot /boot/kernel.bin >> B.OS/boot/grub/grub.cfg
    echo } >> B.OS/boot/grub/grub.cfg
    $(LINKER) $(LDFLAGS) -o $(OUTPUT) $(OBJS)
compile: $(SRCS)
    rm -rf obj/
    mkdir -p obj/

obj/kasm.o: src/kernel.asm
    $(ASSEMBLER) $(ASFLAGS) -o obj/kasm.o src/kernel.asm
obj/kc.o: src/kernel.c
    $(COMPILER) $(CFLAGS) src/kernel.c -o obj/kc.o
```

*inconsistent com a
Fig. 22.*

```

obj/idt.o: src/idt.c
    $(COMPILER) $(CFLAGS) src/idt.c -o obj/idt.o
obj/kb.o: src/kb.c
    $(COMPILER) $(CFLAGS) src/kb.c -o obj/kb.o
obj/isr.o: src/isr.c
    $(COMPILER) $(CFLAGS) src/isr.c -o obj/isr.o
obj/screen.o: src/screen.c
    $(COMPILER) $(CFLAGS) src/screen.c -o obj/screen.o
obj/string.o: src/string.c
    $(COMPILER) $(CFLAGS) src/string.c -o obj/string.o
obj/system.o: src/system.c
    $(COMPILER) $(CFLAGS) src/system.c -o obj/system.o
obj/util.o: src/util.c
    $(COMPILER) $(CFLAGS) src/util.c -o obj/util.o

```

build:

```

# Activate the install xorrr if you do not have it already installed
# sudo apt install xorriso
rm -rf B.OS/boot/grub/
mkdir -p B.OS/boot/grub
echo set default=0 >> B.OS/boot/grub/grub.cfg
echo set timeout=0 >> B.OS/boot/grub/grub.cfg
echo menuentry "B.OS" { >> B.OS/boot/grub/grub.cfg
echo set root='(hd96)' >> B.OS/boot/grub/grub.cfg
echo multiboot /boot/kernel.bin >> B.OS/boot/grub/grub.cfg
echo } >> B.OS/boot/grub/grub.cfg
grub-mkrescue -o B.OS.iso B.OS/

```

all: run build

clean:

```

rm -rf obj/
rm -rf B.OS/
rm B.OS.iso

```

Código 22 – Conteúdo do arquivo Makefile.

Ele usa variáveis para armazenar informações importantes, como compilador, linker e flags de compilação. Vamos analisar brevemente o que cada sessão do makefile realiza tais como suas variáveis, targets e regras de compilation rules.

3.2.1. Variáveis

- **COMPILER:** Armazena o comando do compilador GCC (gcc).
- **LINKER:** Armazena o comando do linker (ld).
- **ASSEMBLER:** Armazena o comando do assembler NASM (nasm).
- **CFLAGS:** Armazena as opções de compilação para o GCC, incluindo a arquitetura (-m32 para 32 bits) e outras flags específicas (-c para compilar, -ffreestanding para compilar sem depender de um ambiente de execução padrão).
- **ASFLAGS:** Armazena as opções de compilação para o NASM, indicando que o formato de saída é ELF de 32 bits (-f elf32).
- **LDFLAGS:** Armazena as opções de linkagem, incluindo a arquitetura (-m elf_i386 para 32 bits) e o arquivo de script de ligação (-T src/link.ld).
- **EMULATOR:** Armazena o comando para iniciar o emulador QEMU (qemu-system-i386).
- **EMULATOR_FLAGS:** Armazena as opções de inicialização para o emulador

- QEMU (-kernel).
- SRCS: Armazena a lista de arquivos-fonte do projeto, incluindo kernel.asm, kernel.c, idt.c, isr.c, kb.c, screen.c, string.c, system.c e util.c localizados no diretório src.
- OBJS: Armazena a lista de arquivos-objeto resultantes da compilação dos arquivos-fonte.
- OUTPUT: Armazena o caminho de saída para o kernel binário gerado (B.OS/boot/kernel.bin).

3.2.2. Alvos (Targets)

- run: Este alvo depende do alvo link. Quando acionado, executa o emulador QEMU para rodar o sistema operacional. Isso é feito através do comando `$(EMULATOR) $(EMULATOR_FLAGS) $(OUTPUT)`.
- link: Este alvo depende do alvo compile e dos objetos `$(OBJS)`. Ao ser acionado, remove diretórios existentes (`rm -rf B.OS`), cria novos diretórios (`mkdir B.OS`, `mkdir B.OS/boot`), e então compilar e linkar os objetos para gerar o executável `$(OUTPUT)`. O comando utilizado é `$(LINKER) $(LDFLAGS) -o $(OUTPUT) $(OBJS)`.
- compile: Este alvo depende dos códigos-fonte `$(SRCS)`. Ao ser acionado, remove e cria o diretório de objetos (`rm obj/ -rf`, `mkdir obj/`), e então compila os códigos-fonte. Cada arquivo-fonte é compilado individualmente.
- build: Este alvo está vinculado a outros alvos como run e build. Ao ser acionado, atualiza o GRUB (um carregador de inicialização) e cria uma imagem ISO utilizando o comando `grub-mkrescue -o B.OS.iso B.OS/`.
- all: Este alvo é frequentemente usado como o alvo padrão quando nenhum alvo específico é fornecido. Neste caso, all é configurado para depender dos alvos run e build. Portanto, ao chamar `make all`, os alvos run e build serão executados.
- clean: Este alvo é usado para limpar os artefatos de compilação e diretórios temporários. Remove os diretórios `obj/` e `B.OS/`, bem como o arquivo ISO `B.OS.iso`.

3.2.3. Regras de Compilação (Compilation Rules)

As regras de compilação especificam como os arquivos-fonte devem ser compilados para gerar os arquivos-objeto correspondentes. Cada regra de compilação é específica para um tipo de arquivo-fonte. No caso do nosso Makefile, temos regras para compilar arquivos Assembly (.asm) e arquivos C (.c). Aqui estão as regras:

- obj/kasm.o: Este objeto é gerado a partir do código-fonte `src/kernel.asm` usando o NASM. Contém o código assembly responsável por funções essenciais do kernel.
- obj/kc.o: É o objeto derivado do código-fonte `src/kernel.c`. Este arquivo representa a lógica principal do kernel escrita em linguagem C.
- obj/idt.o: Gerado a partir do código-fonte `src/idt.c`. Contém implementações relacionadas à Tabela de Descritores de Interrupção (IDT), que é uma estrutura essencial para o tratamento de interrupções.
- obj/kb.o: Este objeto é produzido a partir do código-fonte `src/kb.c`. Contém funcionalidades relacionadas ao teclado.
- obj/isr.o: Gerado a partir do código-fonte `src/isr.c`. Este arquivo contém as rotinas

de serviço de interrupção (ISR), que são chamadas em resposta a interrupções específicas.

- `obj/screen.o`: Objeto derivado de `src/screen.c`. Responsável pela manipulação da tela e exibição de informações.
- `obj/string.o`: Gerado a partir de `src/string.c`. Contém funções relacionadas à manipulação de strings.
- `obj/system.o`: Este objeto é produzido a partir do código-fonte `src/system.c`. Contém código relacionado ao sistema operacional em geral.
- `obj/util.o`: Objeto derivado de `src/util.c`. Contém funções utilitárias diversas.

3.3. Funcionalidades do makefile

Destacamos aqui as características e benefícios proporcionados pelo Makefile no contexto do desenvolvimento do sistema operacional. Aqui estão alguns pontos adicionais para expandir essa seção:

- **Automatização**: Automatiza o processo de compilação.
- **Dependências Explícitas**: Define dependências claras entre alvos.
- **Configurabilidade**: Facilita ajustes centralizando configurações.
- **Limpeza Automática**: Alvo `clean` remove artefatos de compilação.

Em resumo, o Makefile não apenas automatiza o processo de compilação, mas também contribui para a organização, flexibilidade e manutenção eficiente do projeto de desenvolvimento do sistema operacional. Essas funcionalidades são cruciais para garantir um fluxo de trabalho suave e eficaz ao longo do ciclo de desenvolvimento [3, 4].

3.4. Testando na prática

Vamos prosseguir com o teste prático e a demonstração de que o Makefile funciona. Um adendo antes de prosseguirmos, é que, separamos os passos em tópicos para fornecer uma organização clara e facilitar a leitura e entendimento do processo de teste. Cada tópico representa uma etapa específica do teste prático, e essa estrutura serve como um guia para prosseguirmos e lembrarmos as ações necessárias para usar o makefile em nosso projeto e no futuro, portanto, é extremamente importante seguir esta ordem.

3.4.1. PASSO 1 - Compilação e Linkagem

Aqui iremos executar o comando make link para compilar e linkar os arquivos fonte conforme o resultado na figura 24.

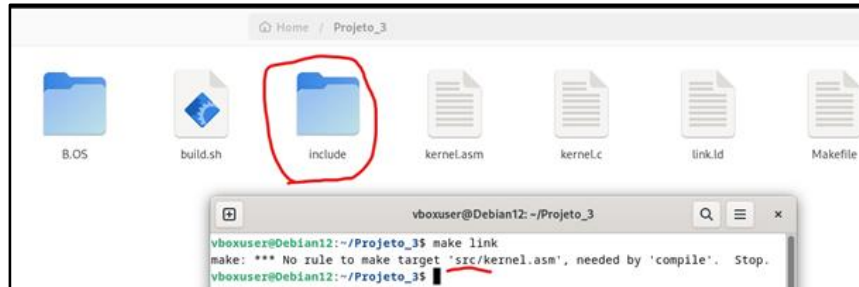


Figura 24 – Resultado da compilação e linkagem.

Infelizmente houve uma falha, mas já identificamos na hora, e o problema provavelmente se trata devido o nome das pastas onde estão nossos sources codes não estarem sendo equivalentes com a do código fornecido pelo professor, sendo assim iremos renomear a pasta “include” -> “src” para que possamos prosseguir e também iremos mover todos os códigos escritos em C, arquivos objetos, arquivos assembly (kernel.asm) e linkers (link.ld) para a pasta “src” para não modificarmos o código conforme foi solicitado de acordo com a Figura 25.



Figura 25 – Movendo arquivos – correção.

Infelizmente tivemos outro erro, mas também é simples de solucionar. O erro está relacionado com a forma como importamos os nossos cabeçalhos, como renomear a pasta include para src, agora precisamos resolver esse problema de path em nossos códigos nessa localização.

Após realizarmos uma busca, encontramos que no arquivo kernel.c estava o erro, sendo assim fizemos a alteração (Figura 26) e executamos novamente e dessa vez deu tudo certo conforme a Figura 27.

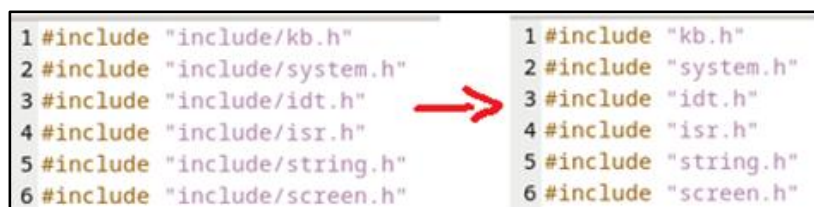


Figura 26 – Renomeando paths includes – correção do arquivo kernel.c.

```

vboxuser@Debian12:~/Projeto_3$ make link
rm obj/ -rf
mkdir obj/
nasm -f elf32 -o obj/kasm.o src/kernel.asm
gcc -m32 -c -ffreestanding src/kernel.c -o obj/kc.o
gcc -m32 -c -ffreestanding src/idt.c -o obj/idt.o
gcc -m32 -c -ffreestanding src/isr.c -o obj/isr.o
gcc -m32 -c -ffreestanding src/kb.c -o obj/kb.o
gcc -m32 -c -ffreestanding src/screen.c -o obj/screen.o
gcc -m32 -c -ffreestanding src/string.c -o obj/string.o
gcc -m32 -c -ffreestanding src/system.c -o obj/system.o
gcc -m32 -c -ffreestanding src/util.c -o obj/util.o
rm -rf B.OS
mkdir B.OS
mkdir B.OS/boot
ld -m elf_i386 -T src/link.ld -o B.OS/boot/kernel.bin obj/kasm.o obj/kc.o obj/idt.o obj/isr.o obj/kb.o obj/screen.o obj/string.o obj/system.o obj/util.o
ld: warning: obj/kasm.o: missing .note.GNU-stack section implies executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
ld: warning: B.OS/boot/kernel.bin has a LOAD segment with R/W permissions

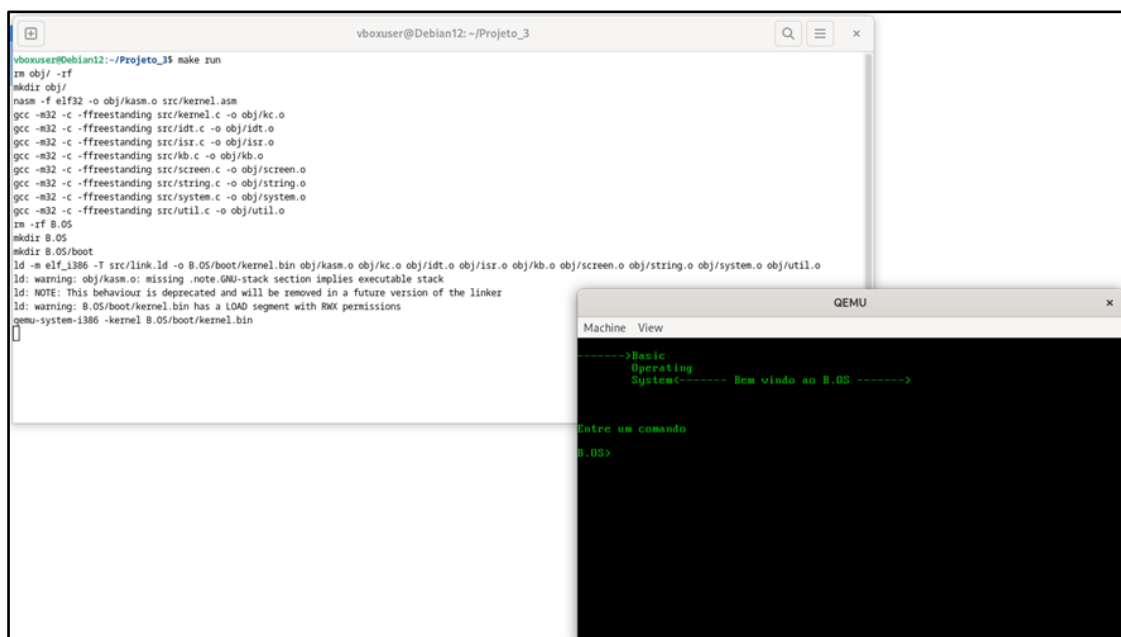
```

Figura 27 – Resultado do comando make link.

Deu tudo certo com esse comando, então basicamente, ele removeu o diretório obj/, criou um novo diretório obj/, compilou os arquivos-fonte em objetos e, finalmente, linkou esses objetos para gerar o executável B.OS/boot/kernel.bin. E aquelas mensagens de aviso do linker como já havíamos falado em outras tarefas, não são necessariamente erros críticos, mas podem ser ignoradas por enquanto.

3.4.2. PASSO 2 - Execução no QEMU

Aqui iremos executar o comando **make run** para **rodar o kernel no emulador QEMU** conforme podemos observar sua execução com êxito na Figura 28.



The image shows a terminal window with the command 'make run' being executed. The output shows the removal of the 'obj/' directory, creation of a new 'obj/' directory, compilation of source files into object files, and linking of the kernel binary. The terminal also shows the execution of 'make run' which runs the kernel in QEMU. A separate QEMU window titled 'QEMU' shows the boot process with the text 'Basic Operating System' and ' Bem vindo ao B.OS '.

```

vboxuser@Debian12:~/Projeto_3$ make run
rm obj/ -rf
mkdir obj/
nasm -f elf32 -o obj/kasm.o src/kernel.asm
gcc -m32 -c -ffreestanding src/kernel.c -o obj/kc.o
gcc -m32 -c -ffreestanding src/idt.c -o obj/idt.o
gcc -m32 -c -ffreestanding src/isr.c -o obj/isr.o
gcc -m32 -c -ffreestanding src/kb.c -o obj/kb.o
gcc -m32 -c -ffreestanding src/screen.c -o obj/screen.o
gcc -m32 -c -ffreestanding src/string.c -o obj/string.o
gcc -m32 -c -ffreestanding src/system.c -o obj/system.o
gcc -m32 -c -ffreestanding src/util.c -o obj/util.o
rm -rf B.OS
mkdir B.OS
mkdir B.OS/boot
ld -m elf_i386 -T src/link.ld -o B.OS/boot/kernel.bin obj/kasm.o obj/kc.o obj/idt.o obj/isr.o obj/kb.o obj/screen.o obj/string.o obj/system.o obj/util.o
ld: warning: obj/kasm.o: missing .note.GNU-stack section implies executable stack
ld: NOTE: This behaviour is deprecated and will be removed in a future version of the linker
ld: warning: B.OS/boot/kernel.bin has a LOAD segment with R/W permissions
qemu-system-i386 -kernel B.OS/boot/kernel.bin

```

Figura 28 – Resultado do comando make run.

E como podemos analisar, o nosso OS executou perfeitamente sem nenhum erro.

3.4.3. PASSO 2 - Execução no QEMU

Aqui iremos executar o comando **make build** para **criar a imagem ISO** conforme podemos observar na Figura 29.

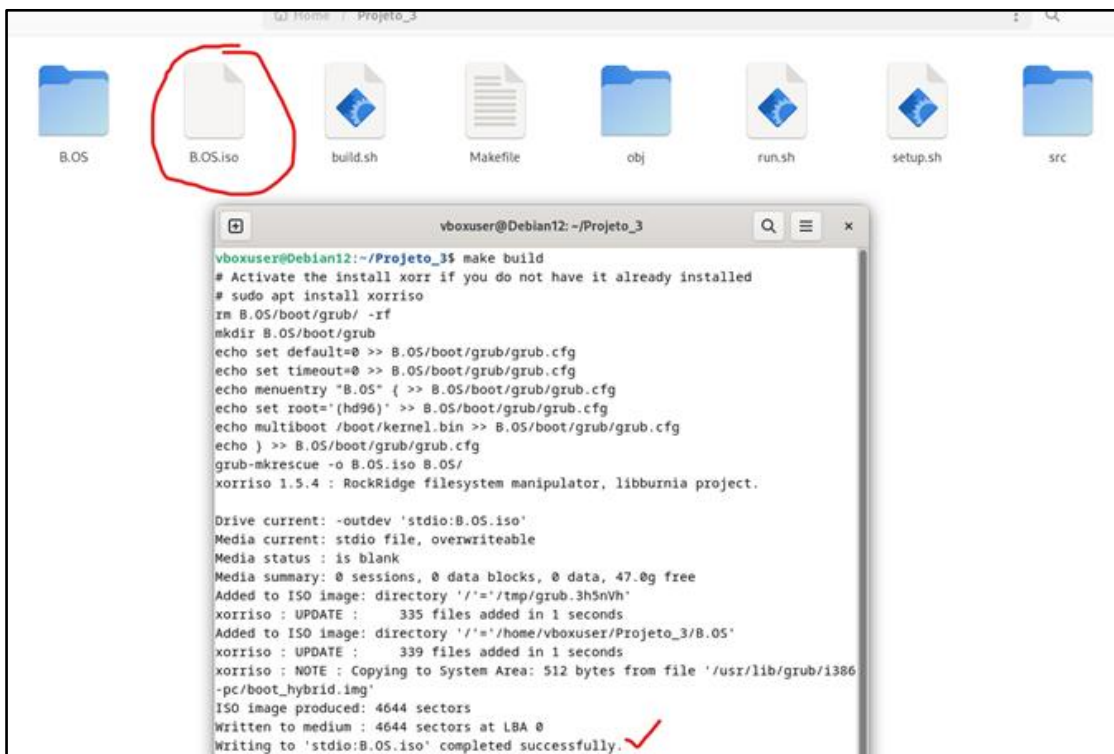


Figura 29 – Resultado do comando make build.

Perfeito, o arquivo ISO foi criado com sucesso, e agora para testarmos essa ISO, teremos de prosseguir com o tópico 3.4.5 (PASSO 5 – Testando ISO), mas antes precisamos realizar a limpeza conforme veremos no tópico 3.4.4 (PASSO 4 - Limpeza do Ambiente)

3.4.4. PASSO 4 - Limpeza do Ambiente

Aqui iremos executar o comando make run para remover todos os artefatos de compilação e reiniciar o ambiente.

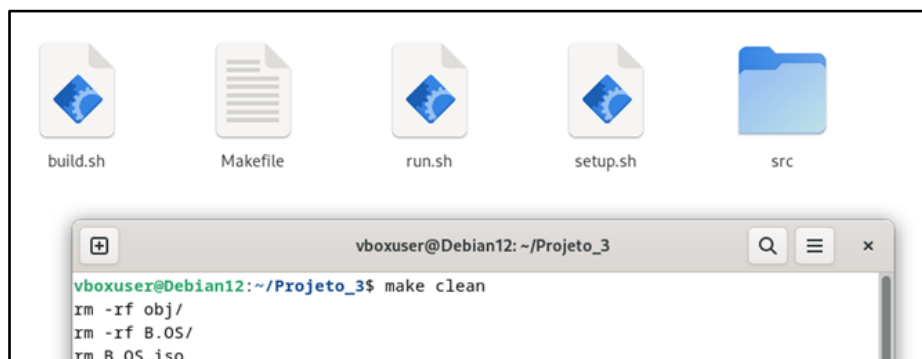


Figura 30 – Resultado do comando make clean.

Deu tudo certo com a remoção, é interessante sempre executar nessa ordem do passo a passo para evitar erros. Por exemplo se fizermos make link para criarmos a pasta B.OS com seus arquivos de inicialização mas executar o comando make clean vai dar erro na última linha pois não irá conseguir excluir o arquivo da ISO pois a mesma ainda não

existe já que é criada usando o comando make build, portanto, é essencial seguir essa ordem: make link, make run, make build e make clean.

3.4.5. PASSO 5 – Testando ISO

Nessa sessão, iremos testar a nossa ISO gerada, porém conforme os conhecimentos adquiridos em todas as aulas, optamos por automatizar a transferência do projeto entre nossas máquinas (convidada e hospedeira) através de um script intuitivo.

Esse script pede ao HOST as seguintes informações:

- Nome do usuario convidado;
- IP do convidado;
- Local de cópia da raiz do projeto no usuario convidado;
- Local de salvamento da pasta no disco do hospedeiro.

O scp_SO2.bat (Código 23) facilita a transferência de arquivos via SCP, permitindo que o usuário insira dinamicamente o nome de usuário, o endereço IP e os caminhos de origem e destino. Essa abordagem flexível se adapta a diferentes configurações de máquinas, sendo útil em ambientes diversos, como laboratórios ou acesso remoto, onde o IP e o usuário podem variar. O script coleta as informações necessárias antes de realizar a cópia dos arquivos, tornando-o eficiente e versátil. Além disso, é oferecido um exemplo para não esquecer de como se utiliza o comando em cada solicitação. Por conta de ser algo pessoal esse script não será compartilhado e nem anexado com o arquivo final pois está fora do escopo da tarefa.

```
@echo off
set /p usuario="Digite o nome de usuario (GuestEx > vboxuser):"
set /p ip="Digite o endereço IP (GuestEx > 127.0.0.1):"
set /p origem="Digite o caminho do arquivo de origem (GuestEx > /home/vboxuser/Projeto_3/):"
set /p destino="Digite o caminho de destino (HostEx > E:/):"

scp -r %usuario%@%ip%:%origem% %destino%

pause
```

Código 23 – Conteúdo arquivo scp_SO2.bat.

```
> .\scp_S02.bat
Digite o nome de usuario (GuestEx > vboxuser):vboxuser
Digite o endereço IP (GuestEx > 127.0.0.1):127.0.0.1
Digite o caminho do arquivo de origem (GuestEx > /home/vboxuser/Projeto_3/):/home/vboxuser/Downloads/Projeto_3/
Digite o caminho de destino (HostEx > E:/):D:/
vboxuser@127.0.0.1's password:
util.c 100% 665 216.5KB/s 00:00
idt.c 100% 495 241.7KB/s 00:00
scancode to ascii.gif 100% 19KB 6.2MB/s 00:00
system.c 100% 262 127.9KB/s 00:00
string.c 100% 492 240.2KB/s 00:00
isr.h 100% 601 293.5KB/s 00:00
system.h 100% 142 69.3KB/s 00:00
isr.c 100% 4062 989.2KB/s 00:00
kernel.asm 100% 556 271.5KB/s 00:00
types.h 100% 419 409.2KB/s 00:00
link.ld 100% 133 64.9KB/s 00:00
idt.h 100% 506 247.1KB/s 00:00
screen.h 100% 981 239.5KB/s 00:00
kb.h 100% 103 50.3KB/s 00:00
string.h 100% 146 71.3KB/s 00:00
kernel.c 100% 639 312.0KB/s 00:00
screen.c 100% 4492 2.1MB/s 00:00
kb.c 100% 7445 1.4MB/s 00:00
util.h 100% 206 100.6KB/s 00:00
B.OS.iso 100% 9288KB 50.1MB/s 00:00
kernel.bin 100% 16KB 7.9MB/s 00:00
grub.cfg 100% 90 44.0KB/s 00:00
Makefile 100% 1845 450.4KB/s 00:00
run.sh 100% 56 27.3KB/s 00:00
build.sh 100% 863 842.6KB/s 00:00
string.o 100% 1352 660.2KB/s 00:00
system.o 100% 1224 597.6KB/s 00:00
kasm.o 100% 512 166.7KB/s 00:00
kb.o 100% 5560 2.7MB/s 00:00
isr.o 100% 7972 3.8MB/s 00:00
kc.o 100% 1848 902.3KB/s 00:00
screen.o 100% 3472 1.1MB/s 00:00
util.o 100% 1492 728.5KB/s 00:00
idt.o 100% 1388 1.3MB/s 00:00
setup.sh 100% 547 267.1KB/s 00:00
Press any key to continue . . .
```

Figura 31 – Transferência via scp realizada com sucesso.

Agora que temos o projeto em mãos da máquina hospedeira, iremos alterar a ISO no disco óptico da nossa máquina virtual chamada de BasicOS utilizando a ferramenta de gerenciamento de mídias virtuais do virtualbox.

Sendo assim, primeiramente iremos remover a ISO antiga (Figura 32) para podermos adicionar depois a nova.

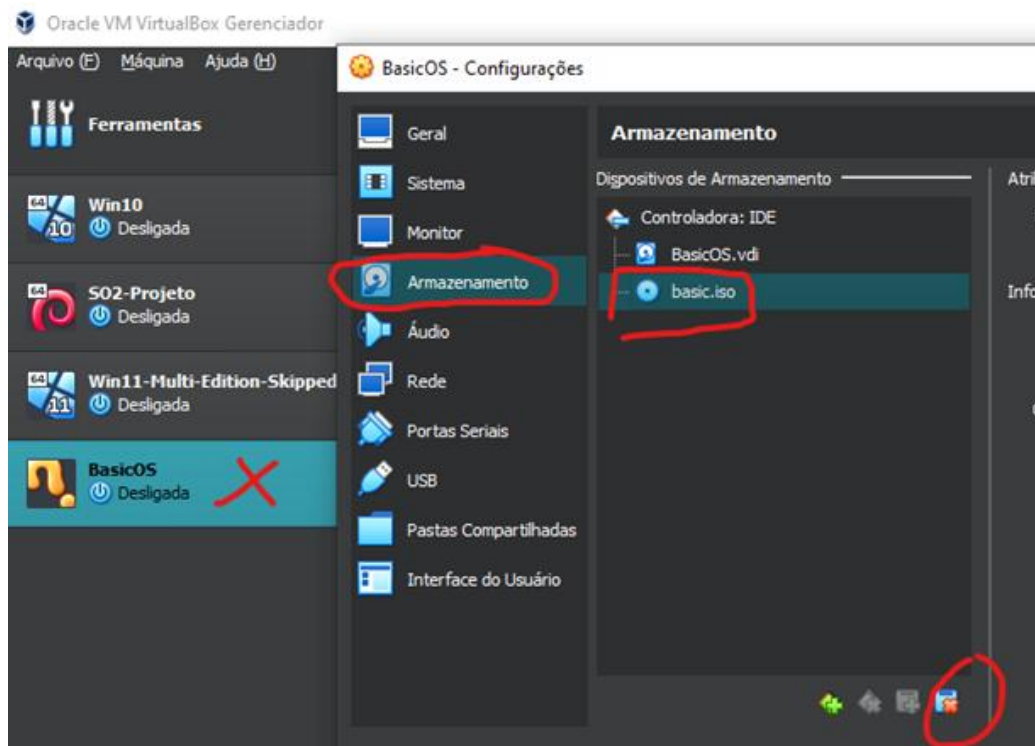


Figura 32 – Remover a ISO antiga do BasicOS.

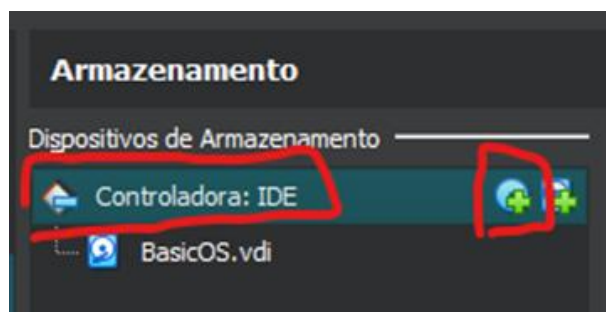


Figura 33 – Selecionar ISO nova/atualizada – Parte 1.

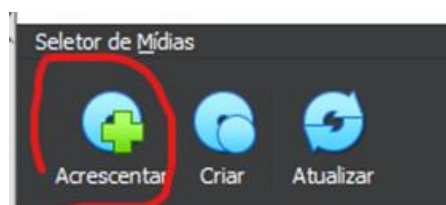


Figura 34 – Selecionar ISO nova/atualizada – Parte 2.

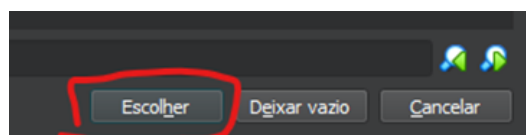


Figura 35 – Selecionar ISO nova/atualizada – Parte 3.

Agora a ISO já está trocada com sucesso. Entretanto, precisamos limpar as ISOs antigas ainda, pois apenas acrescentamos mas não excluimos. Sendo assim iremos usar o atalho **ctrl+d** para entrarmos no gerenciamento de mídia conforme os passos

demonstrados na Figura 36.

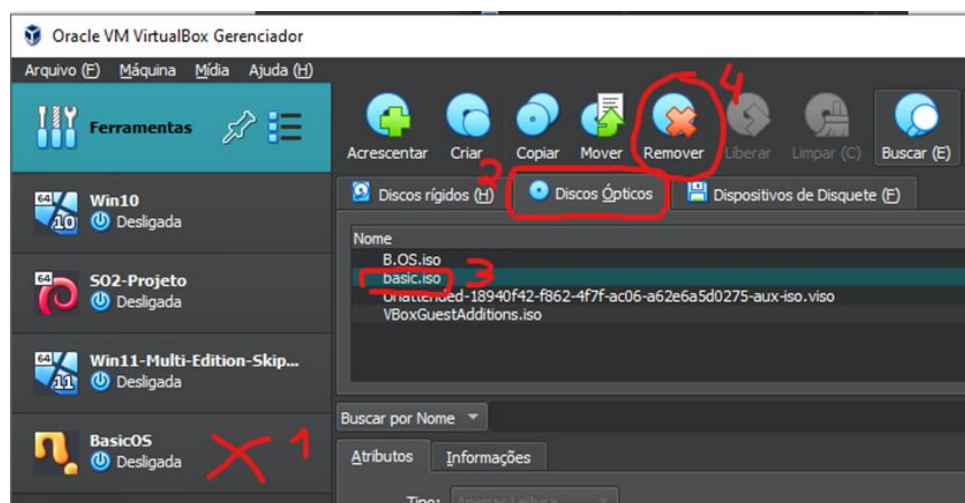


Figura 36 – Remover ISO antiga.

Pronto, agora que deixamos tudo organizado basta testar a ISO conforme vemos na Figura 37 para finalizarmos esta tarefa.

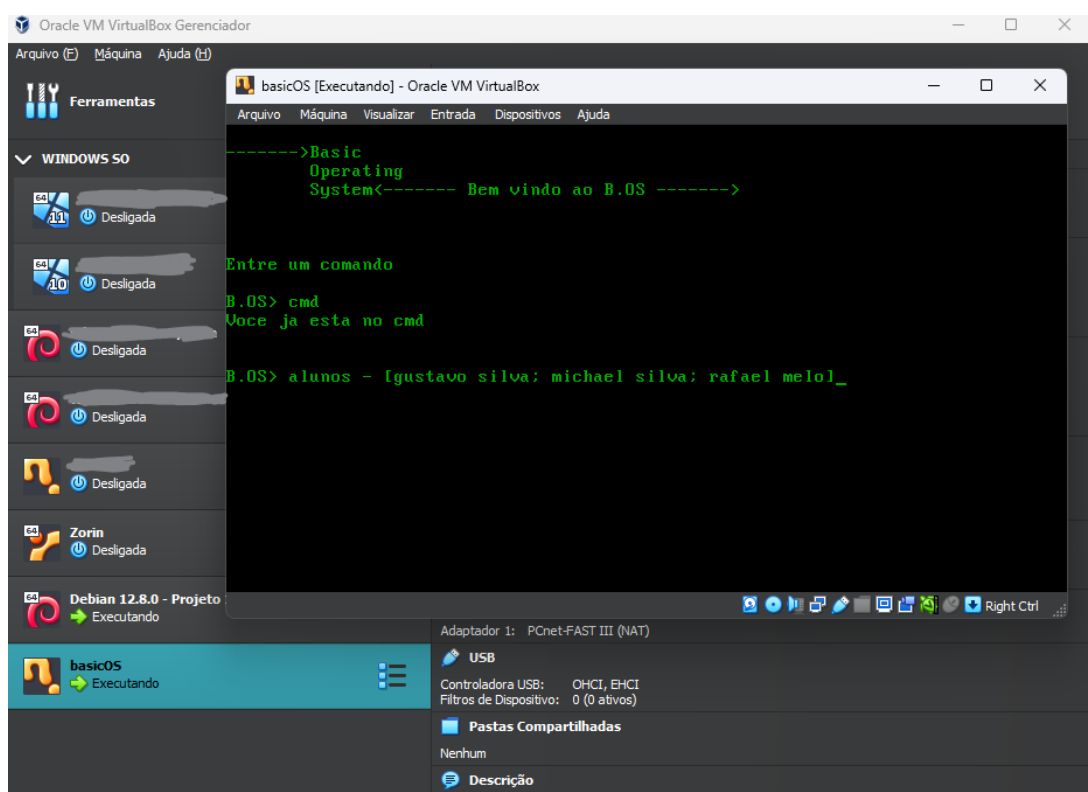


Figura 37 – ISO funcionando perfeitamente.

4. Tarefa 4: construção de um shell simples

(06/02/2025 a 09/02/2025).

5. Considerações finais

(06/02/2025 a 09/02/2025).

6. Referências

[1] Dalton Matsuo Tavares, (2025) “Projeto 3 – Projeto de um S.O. (BOS - Basic Operating System)”, material disponibilizado no google classroom e SIGAA, Dezembro.

[2] Milliseconds, (2023) “Keyboard Scan Codes”, <https://www.millisecond.com/support/docs/v5/html/language/scancodes.htm>, Dezembro.